



COURSE SLIDES · NON-WSQ COURSE

Linux Masterclass

Course Code: C579

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.0 · 1 October 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Dr Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE
Principal Trainer, Tertiary Infotech Academy Pte Ltd

EXPERTISE
Linux system administration, shell scripting, DevOps and software engineering.

DELIVERS
Courses on Linux, cloud computing, data science, AI and secure software engineering.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

- Your name and organisation / role.
- Your experience with Linux, Unix or the command line (if any).
- What you want to be able to configure or automate after this course.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

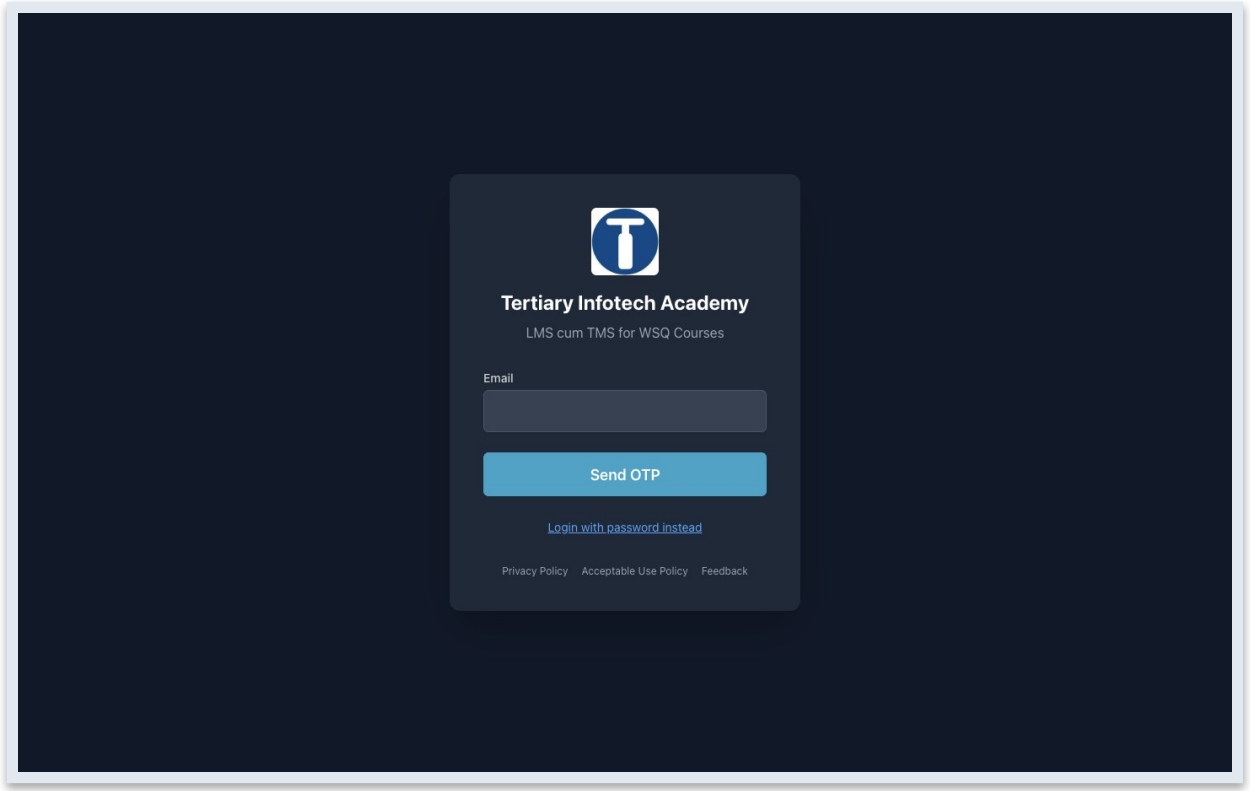
Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. Full attendance — attend both days to get the most from the course.

© 2026 Tertiary Infotech Academy Pte Ltd

LMS / TMS PORTAL

Download Course Material



Log in with your enrolment email → open your course → download the slides and Learner Guide.

https://lms-tms.tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 2 Days, 7.5 hours/day

Day 1

- Day 1 — Linux Fundamental & Shell Scripting
  - Welcome & introductions
  - Topic 1: Linux Fundamental (Labs 1–4)
  - Lunch (30 min)
  - Topic 2: Linux Shell Script (Labs 5–12)
  - Day 1 recap and Q&A

Day 2

- Day 2 — Networking Configuration & Security
  - Day 1 recap
  - Topic 3: Linux Networking Configuration (Labs 13–15)
  - Lunch (30 min)
  - Topic 4: Linux Security (Labs 16–17)
  - Course wrap-up, feedback and Q&A
  - Daily timing: 9:30am–5:30pm · 30-min lunch

© 2026 Tertiary Infotech Academy Pte Ltd

FOUR TOPICS · 17 HANDS-ON LABS

Course Outline

1

Topic 1 · Linux Fundamental

Linux components, distributions, file system, users and permissions.

2

Topic 2 · Linux Shell Script

Shell environment, text processing, and Bash scripting from variables to functions.

3

Topic 3 · Linux Networking Configuration

Processes, cron, packages, service deployment and network configuration tests.

4

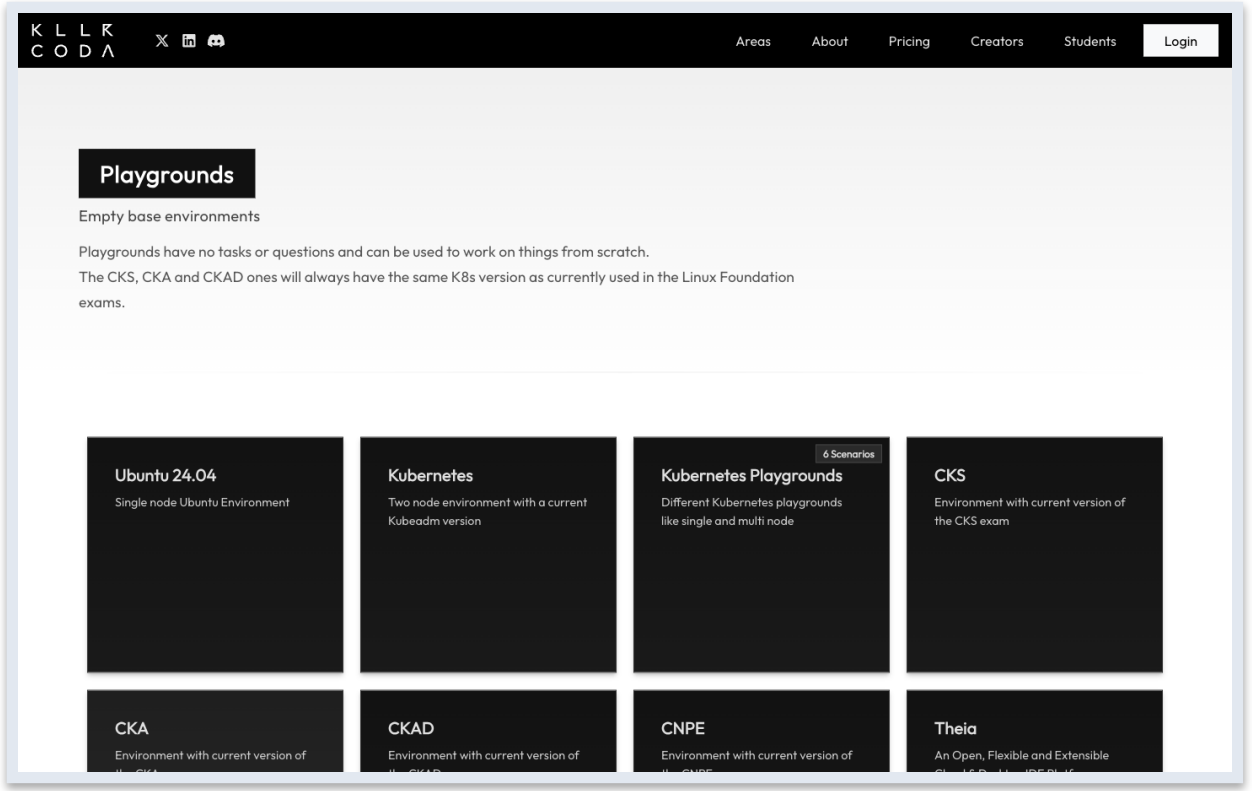
Topic 4 · Linux Security

Security principles, least privilege, firewall, hardening and audit scripts.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON ENVIRONMENT

Your Lab Platform — KillerCoda



Every lab runs in the free browser-based Ubuntu playground — no installation on your laptop.

https://killercoda.com/playgrounds/scenario/ubuntu

© 2026 Tertiary Infotech Academy Pte Ltd

01

TOPIC 01

Linux Fundamental

Linux components · distributions · file system · users and permissions

© 2026 Tertiary Infotech Academy Pte Ltd

KEY CONCEPTS

Key Concepts — Linux Fundamental

1

Linux OS & kernel

Linux is a free, open-source, Unix-like OS; the kernel plus apps and tools form a distribution.

2

Distributions

RHEL, Fedora, CentOS, AlmaLinux, Ubuntu, Debian, SUSE — pick by need, support and package system.

3

Ways to run Linux

Virtual machine, native install, live boot, cloud service, WSL — or a browser playground like KillerCoda.

4

File system hierarchy

One tree from / — /bin, /etc, /home, /opt, /tmp, /usr, /var; everything is a file.

5

Users & superuser

Regular, system, service and group accounts; root is all-powerful — use su/sudo with care.

6

Permissions

Read/write/execute for user, group and other — set with chmod (alpha or octal), chown and umask.

© 2026 Tertiary Infotech Academy Pte Ltd

FOUNDATIONS

What is Linux?

1

An operating system

The software layer that manages your hardware and runs applications.

2

A kernel

At its core, 'Linux' is the kernel created by Linus Torvalds in 1991; v1.0 shipped in 1994.

3

FOSS

Free and open-source software — anyone can inspect, modify and redistribute the code.

4

Unix-like

Linux follows Unix design: everything is a file, small tools, plain-text pipes.

© 2026 Tertiary Infotech Academy Pte Ltd

Why Linux?

1. Runs on many hardware platforms — servers, desktops, Raspberry Pi, phones.
2. Small footprint and efficient on modest hardware.
3. Stable, reliable and secure — powers most of the internet's servers.
4. Free and open source, with a huge free-software ecosystem.
5. Choice and freedom — hundreds of distributions to fit any need.
6. The native home of the shell and shell scripting.

© 2026 Tertiary Infotech Academy Pte Ltd

FROM GNU TO TODAY

History of Linux

1. 1984 — Stallman starts the GNU project: a free Unix-like OS (tools, compilers, libraries)
2. 1991 — Linus Torvalds writes the Linux kernel and combines it with the GNU toolchain
3. 1994 — Linux 1.0 released; distributions begin to package kernel + apps
4. Today — Linux powers servers, clouds, Android, supercomputers and embedded devices

© 2026 Tertiary Infotech Academy Pte Ltd

ANATOMY

Kernel vs Distribution

Kernel

- **The kernel is the core**
  - Manages CPU, memory, devices and processes
  - Kernel + apps + tools = a distribution (distro)
  - A distro is Linux packaged, integrated and tested for users

Distribution

- **Each distro differs by**
  - Package management system
  - Release cadence and support model
  - Design philosophy and target audience
  - Community or commercial backing

© 2026 Tertiary Infotech Academy Pte Ltd

THE DISTRO LANDSCAPE

Popular Linux Distributions

1

Red Hat family

RHEL, Fedora, CentOS, AlmaLinux, Rocky — RPM packages, dnf/yum.

2

Debian family

Debian, Ubuntu, Linux Mint, Kali — DEB packages, apt.

3

SUSE family

SUSE Linux Enterprise Server (SLES) and openSUSE — zypper.

4

Choosing one

Desktop needed? Ubuntu/Mint/Fedora. Server? Ubuntu Server or Debian. Same software may differ by name — Apache is apache2 on Ubuntu, httpd on Red Hat.

© 2026 Tertiary Infotech Academy Pte Ltd

SIX OPTIONS

Ways to Run Linux

1

Virtual machine

VirtualBox, VMware, Hyper-V — an isolated, protected OS inside your computer.

2

Native install

Full performance on real hardware — even a Raspberry Pi; replaces the existing OS.

3

Live boot

Boot from USB without touching the disk — easy, but slower and non-persistent.

4

Cloud service

AWS, Azure, Google Cloud, Alibaba — great for SSH text access, no desktop.

5

WSL

Windows Subsystem for Linux — distros from the Microsoft Store on Windows 10/11.

6

Browser playground

KillerCoda gives a real Ubuntu sandbox in your browser — our course platform.

© 2026 Tertiary Infotech Academy Pte Ltd

RELEASE COMPONENTS

Identify Your Linux Release

```
# who am I logged in as?
$ whoami

# kernel release and architecture
$ uname -a

# distribution name and version
$ lsb_release -a

# full release record
$ cat /etc/os-release

# which shell am I running?
$ echo $SHELL
```

A release = kernel + distribution + shell + packages. These commands identify each component.

© 2026 Tertiary Infotech Academy Pte Ltd

THE FILE SYSTEM

Everything in Linux hangs off one tree — the root directory /.

The file system is hierarchical: one root, with standard directories whose purpose every admin knows.

© 2026 Tertiary Infotech Academy Pte Ltd

FILE SYSTEM HIERARCHY

Common Directories

1. **/**
   'Root' — the top of the file system hierarchy.
2. **/bin**
   Binaries and other executable programs.
3. **/etc**
   System configuration files.
4. **/home**
   Users' home directories.
5. **/opt**
   Optional or third-party software.
6. **/tmp**
   Temporary space, typically cleared on reboot.
7. **/usr**
   User-related programs and read-only data.
8. **/var**
   Variable data — most notably log files in /var/log.

© 2026 Tertiary Infotech Academy Pte Ltd

PATHS & SHORTCUTS

Navigating the Tree

```
# where am I?
$ pwd

# absolute path
$ cd /var/log

# home shortcut (tilde expansion)
$ cd ~

# up one level · this directory
$ cd ..    # '..' parent · '.' current

# visual tree, directories only
$ tree -d -L 2 /var
```

Absolute paths start at /; relative paths start where you stand. ~ expands to your home directory.

© 2026 Tertiary Infotech Academy Pte Ltd

THE MULTIUSER MODEL

User Accounts on Linux

1

Superuser (root)

The most powerful account — can perform any action and access any file.

2

Regular user

Standard account for everyday tasks with limited privileges.

3

System account

Runs system services and daemons; generally no login shell.

4

Group account

Manages a collection of users to control access to resources.

5

Service account

Runs a specific service or application with limited privileges.

6

Guest account

Temporary access with minimal privileges.

© 2026 Tertiary Infotech Academy Pte Ltd

PRIVILEGE

Root, the Superuser

Root

- **Root is all-powerful**
  - Normal accounts do a subset of what root can
  - Needed to install, start or stop system software
  - Access is restricted to administrators
  - Be careful — root can destroy the system

su & sudo

- **Borrowing root safely**
  - su [user] — switch user; '-' loads their environment
  - sudo command — run one command as root
  - sudo -l — list what you may run
  - sudo su - — become root with root's environment

© 2026 Tertiary Infotech Academy Pte Ltd

MEET YOUR INTERPRETER

What is the Shell?

1

Command interpreter

A program that accepts your commands and executes them — the default interface to Linux.

2

Many shells

sh (Bourne 1977), csh/tcsh (1979), ksh (1982), bash (1987), zsh (1990) — bash is the Linux default.

3

CLI vs GUI

The command line is more powerful and always available; servers ship without a GUI.

4

Check yours

echo $SHELL shows your login shell; environment variables are UPPERCASE with a $ prefix.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Linux Fundamental

Labs 1–2

- Getting Started with the KillerCoda Ubuntu Playground
- Exploring the Linux File System

Labs 3–4

- Working with Files, Directories and Links
- Users, Ownership and Permissions

—

- —

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Getting Started with the KillerCoda Ubuntu Playground

LAB 1

Launch the free browser-based KillerCoda Ubuntu playground, explore the terminal, and identify the components of the running Linux release: the kernel version, the distribution and the shell.

You'll build

A working Ubuntu lab environment with the course exercise files cloned from GitHub.

Tools: KillerCoda Ubuntu playground, bash, git

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 1

```
# Open the KillerCoda Ubuntu playground in your browser and …
$ https://killercoda.com/playgrounds/scenario/ubuntu
# Confirm who you are logged in as
$ whoami
# Identify the kernel release and system architecture
$ uname -a
# Identify the Ubuntu distribution and release
$ lsb_release -a
# Inspect the distribution details recorded in the os-release …
$ cat /etc/os-release
# Check which shell you are running
$ echo $SHELL
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · VERIFY

Getting Started with the KillerCoda Ubuntu Playground

✅ Test it

uname -a shows a Linux kernel, lsb_release -a shows Ubuntu, and sysinfo.sh prints a system report with the kernel, distro, shell, uptime and disk usage.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Exploring the Linux File System

LAB 2

Navigate the Linux file system hierarchy from the root directory down: identify what lives in /bin, /etc, /home, /var and friends, use absolute and relative paths, and reveal hidden files.

You'll build

A mental map of the Linux directory tree and fluency with pwd, cd, ls and tree.

Tools: bash, coreutils, tree

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 2

```
# Print your current (home) directory
$ pwd

# List the top of the file system hierarchy
$ ls /

# Look at system configuration files in /etc
$ ls /etc | head

# Look at log files in /var/log
$ ls /var/log

# Change into /etc with an absolute path, then return home …
$ cd /etc && pwd && cd ~ && pwd

# Move up one level with the .. shortcut
$ cd .. && pwd && cd ~
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · VERIFY

Exploring the Linux File System

✅ Test it

You can explain what /bin, /etc, /home, /tmp, /usr and /var hold, and ls -la ~ reveals hidden dot files such as .bashrc.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Working with Files, Directories and Links

LAB 3

Create and manage files and directories with touch, mkdir, cp, mv and rm; generate file sets with brace expansion; edit with nano; and compare hard links against symbolic links.

You'll build

A working directory of files you created, renamed, moved and linked — including a hard link that survives deletion of its original and a symlink that breaks.

Tools: bash, coreutils, nano

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 3

```
# Make a working directory and change into it
$ mkdir ~/demo && cd ~/demo

# Create an empty file with touch and check it
$ touch new_file.txt && ls -l

# Create six files at once with brace expansion
$ touch apple_{A,B}{1..3}

# Verify the expansion created apple_A1..apple_B3
$ ls apple_*

# Edit new_file.txt with nano — type a line, then Ctrl-X, y …
$ nano new_file.txt

# Copy a file, then rename it with mv
$ cp new_file.txt copy.txt && mv copy.txt renamed.txt
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · VERIFY

Working with Files, Directories and Links

✅ Test it

cat lorem2 still shows the text after lorem.txt is deleted, while cat lorem3 reports 'No such file or directory' — you can explain why.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Users, Ownership and Permissions

LAB 4

Work with the multiuser model: switch identities with su/sudo, read ls -l permission strings, and control access with chmod (alphabetic and octal), chown and umask.

You'll build

A fruit.txt file whose permissions you set to rw-r--r-- and rwxrwxrw-, plus a file owned by root.

Tools: bash, sudo, coreutils

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 4

```
# Confirm your identity and list what sudo lets you do
$ whoami && sudo -l | head

# Copy the fruit.txt exercise file to your home directory
$ cp ~/course/labs/lab-04-permissions/fruit.txt ~/fruit.txt && cd ~

# Decode the permission string in a long listing
$ ls -l fruit.txt

# Give user+group execute with alphabetic notation, then …
$ chmod ug+x fruit.txt && ls -l fruit.txt && chmod ug-x fruit.txt

# Set exact permissions rw-/r--/r-- with alphabetic notation
$ chmod u=rw,g=r,o=r fruit.txt && ls -l fruit.txt

# Set rwxrwxrw- using octal notation (776)
$ chmod 776 fruit.txt && ls -l fruit.txt
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · VERIFY

Users, Ownership and Permissions

✅ Test it

ls -l shows -rw-r--r-- after the alphabetic chmod and -rwxrwxrw- after chmod 776; the root-owned file refuses your write until you chown it back.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Linux Fundamental

- You can now: Analyse the components of a Linux release — kernel, shell and distribution
- You can now: Analyse how a Linux system organises its release components on disk
- You can now: Coordinate release files — create, copy, move, remove and link them
- You can now: Coordinate access to release components with users, groups and permissions

© 2026 Tertiary Infotech Academy Pte Ltd

02

TOPIC 02

Linux Shell Script

Shell environment · text processing · Bash scripting: variables, conditionals, loops, functions

© 2026 Tertiary Infotech Academy Pte Ltd

KEY CONCEPTS

Key Concepts — Linux Shell Script

1

The shell

A command-line interpreter (bash, zsh, ksh, csh…) — the default, scriptable interface to Linux.

2

Environment

Aliases, environment variables, PATH and PS1 customise the shell; persist them in ~/.bashrc.

3

Redirection & pipes

> >> < 2> redirect streams; | chains commands; /dev/null discards output.

4

Text power tools

grep with regular expressions, sort, cut, uniq, head/tail, diff — process any text stream.

5

Bash scripts

A script = shebang + commands; make it executable with chmod +x and run it with ./script.sh.

6

Script constructs

Variables, read input, positional arguments, arithmetic, if/test/case, for/while/until, functions, exit codes.

© 2026 Tertiary Infotech Academy Pte Ltd

EVERY UNIX COMMAND LINE

Command Structure

1

COMMAND — the program to run (echo, ls, grep)

2

OPTIONS — switches that modify behaviour (-l, -a, --help)

3

ARGUMENTS — what to act on (files, directories, text)

4

Example: ls -lah Desktop = ls -l -a -h Desktop

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR FIRST EIGHT

Basic Linux Commands

1. **ls**
   Lists directory contents.
2. **cd**
   Changes the current directory.
3. **pwd**
   Displays the present working directory.
4. **cat**
   Concatenates and displays files.
5. **echo**
   Displays arguments to the screen.
6. **man**
   Displays the online manual.
7. **exit**
   Exits the shell or current session.
8. **clear**
   Clears the screen.

© 2026 Tertiary Infotech Academy Pte Ltd

Getting Help

```
# read the manual (q to quit; Space = page, g/G = top/bottom)
$ man ls

# search the man pages by keyword
$ man -k copy

# quick help from the command itself
$ ls --help   # or -h

# where does a command live?
$ which nano
```

Enter moves one line, Space one page, g top, G bottom, q quits the pager.

© 2026 Tertiary Infotech Academy Pte Ltd

BEYOND PLAIN ls

Listing Files — ls Power Options

```
# long format, all files, human sizes
$ ls -lah

# sort by time · reverse
$ ls -t    ls -r    ls -latr

# reveal types: / dir · @ link · * executable
$ ls -F

# recursive listing · directory itself
$ ls -R    ls -d

# decode: -rw-r--r-- 1 owner group size date name
$ ls -l fruit.txt
```

ls -l shows permissions, links, owner, group, size, modified time and name.

© 2026 Tertiary Infotech Academy Pte Ltd

TWO KINDS OF LINK

Hard Links vs Symbolic Links

Hard

- **Hard link — ln file shortcut**
  - References the file's data on disk
  - Survives moves of the original
  - Survives deletion of the original
  - Cannot span file systems or link directories

Symbolic

- **Symbolic link — ln -s file symlink**
  - References a file or directory path
  - Breaks if the original is moved
  - Breaks if the original is deleted
  - Used for shortcuts and 'current version' pointers

© 2026 Tertiary Infotech Academy Pte Ltd

NAME FILES WELL

File Naming Rules

1. Maximum of 255 characters.
2. Avoid special characters such as / \ & % and spaces.
3. Use A–Z, a–z, 0–9, period, underscore and hyphen.
4. Names are case sensitive: Notes.txt ≠ notes.txt.
5. Prefer underscores/hyphens/CamelCase over spaces; quote or escape (\ ) if you must.
6. Extensions (.txt, .sh) are optional but helpful.

© 2026 Tertiary Infotech Academy Pte Ltd

THE PERMISSION MODEL

Permissions — r w x

1

r = read

View a file's contents; list a directory.

2

w = write

Modify a file; create/delete entries in a directory.

3

x = execute

Run a file as a program; enter (cd into) a directory.

4

Three audiences

u = user (owner), g = group, o = other — shown as rwxrwxrwx in ls -l.

© 2026 Tertiary Infotech Academy Pte Ltd

Changing Permissions & Ownership

```
# alphabetic notation
$ chmod u=rwx,g=rw,o=r file    chmod ug+x file
# octal notation: r=4 w=2 x=1
$ chmod 764 file    # rwx rw- r--
# recursive on a directory tree
$ chmod -R 755 mydir
# change owner (and group) — needs root
$ sudo chown user:group file
# change group only
$ chgrp developers file
# default permissions mask (666/777 minus mask)
$ umask    # commonly 022 → files 644
```

Directory permissions gate the files inside — if file permissions look right, check the directories above.

© 2026 Tertiary Infotech Academy Pte Ltd

Editors — nano and vi

nano

- **nano — simple and friendly**
  - nano file — edit in place; Ctrl-X, y, Enter saves
  - On-screen key hints; ideal for quick edits
  - Also: touch creates empty files; gedit is graphical

vi

- **vi / vim — powerful, modal**
  - Esc = command mode · i/a = insert · : = line mode
  - Navigate: h j k l · w b · ^ $ · dd delete · yy yank · p paste
  - :w save · :q quit · :wq! save+quit · u undo · /find

© 2026 Tertiary Infotech Academy Pte Ltd

INSPECTING CONTENT

Reading Files

1

cat

Print the whole file (or concatenate several).

2

more / less

Page through output — less lets you scroll back.

3

head

First 10 lines; head -15 file changes the count.

4

tail

Last 10 lines; tail -f follows the file in real time — perfect for logs.

5

file

Reports what type of data a file holds.

6

strings

Extracts printable text from binary files.

© 2026 Tertiary Infotech Academy Pte Ltd

POWER TOOLS

Finding, Sorting and Cutting

```
# find files by name pattern (* ? [])
$ find ~/Documents -name '*.csv'

# sort text — by field, reverse, unique
$ sort -k 2 -r -u file.txt

# cut a delimited field
$ cut -d',' -f2 us_presidents.csv

# count lines, words, characters
$ wc -l fruit.txt

# compare files
$ diff original.txt revised.txt   # sdiff side-by-side
```

Small tools that each do one job well — then combine them with pipes.

© 2026 Tertiary Infotech Academy Pte Ltd

Redirection

1

>

Redirect stdout to a file — overwrites (truncates) existing contents.

2

>>

Redirect stdout to a file — appends to existing contents.

3

<

Feed a file into a command's stdin.

4

2> and 2>&1

Redirect stderr to a file; combine stderr with stdout.

5

| (pipe)

Connect one command's output to the next command's input.

6

/dev/null

The null device — redirect here to discard output.

© 2026 Tertiary Infotech Academy Pte Ltd

THE UNIX WAY

Redirection & Pipes in Action

```
# overwrite, then append
$ sort fruit.txt > sorted.txt    echo mango >> sorted.txt

# join files into one
$ cat new.txt newer.txt > joined.txt

# file → command
$ wc < fruit.txt

# pipeline: sort then de-duplicate
$ sort fruit.txt | uniq

# save every process to a file
$ ps aux > processes.txt

# page long output
$ cat lorem_ipsum.txt | less
```

Pipes chain small tools into pipelines — the heart of shell productivity.

© 2026 Tertiary Infotech Academy Pte Ltd

THE SEARCH WORKHORSE

grep — Search with Patterns

```
# find lines matching a pattern
$ grep apple fruit.txt

# ignore case · whole word · count · invert
$ grep -i Apple f.txt   grep -w apple f.txt   grep -c apple f.txt   grep -v apple f.txt

# line numbers · matching files · recursive
$ grep -n apple f.txt   grep -l apple *.txt   grep -R apple dir/

# search a stream through a pipe
$ ps aux | grep bash

# highlight matches
$ grep --color=auto lorem lorem_ipsum.txt
```

grep displays lines matching a pattern — from files or from a pipe.

© 2026 Tertiary Infotech Academy Pte Ltd

PATTERN LANGUAGE

Regular Expressions

```
# any single character
$ . → grep 'a..le' fruit.txt

# character class
$ [cp] → grep 'ea[cp]' fruit.txt

# anchors: start ^ and end $
$ grep '^p' fruit.txt    grep 'berry$' fruit.txt

# one-or-more (extended)
$ grep -E 'ap+le' fruit.txt

# alternation
$ grep -E 'apple|pear' fruit.txt

# class + anchor together
$ grep '^[sp]' fruit.txt   # starts with s or p
```

A regular expression describes a text pattern — grep, sed and awk all speak it.

© 2026 Tertiary Infotech Academy Pte Ltd

Anatomy of a Shell Script

1

#!/bin/bash — the shebang tells Linux which interpreter runs the file

2

Commands + variables + logic — anything you can type, you can script

3

chmod +x script.sh — make it executable

4

./script.sh — run it (or bash script.sh)

© 2026 Tertiary Infotech Academy Pte Ltd

SCRIPT BUILDING BLOCKS 1

Variables, Input & Arithmetic

```
# assign (no spaces around =) and expand
$ name='Alfred'    echo "Hello $name"
# command substitution
$ today=$(date +%F)
# read user input
$ read -p 'Your name? ' name
# positional parameters
$ $0 script · $1 $2 args · $# count · $@ all
# integer arithmetic
$ sum=$(( $1 + $2 ))    echo $((7*6))
```

Double quotes expand $variables; single quotes print literally.

© 2026 Tertiary Infotech Academy Pte Ltd

Conditionals — if, test and case

```
# numeric tests: -eq -ne -gt -lt -ge -le
$ if [ "$n" -gt 0 ]; then echo positive; fi
# string tests
$ [ "$s" = 'linux' ]    [ -z "$s" ]  # empty
# file tests
$ [ -e path ] exists · [ -f ] file · [ -d ] dir · [ -x ] executable
# full branch
$ if …; then …; elif …; then …; else …; fi
# multi-way selection
$ case $fruit in apple|pear) … ;; *) … ;; esac
```

test (the [ ] brackets) returns an exit status the if statement branches on.

© 2026 Tertiary Infotech Academy Pte Ltd

SCRIPT BUILDING BLOCKS 3

Loops — for, while and until

```
# over a word list
$ for fruit in apple banana cherry; do echo $fruit; done

# over a range
$ for i in {1..5}; do echo $i; done

# over files
$ for f in *.txt; do wc -l "$f"; done

# while a condition holds
$ while [ $i -gt 0 ]; do (( i-- )); done

# until a condition becomes true
$ until [ $i -gt 5 ]; do (( i++ )); done

# flow control
$ continue skips an iteration · break exits the loop
```

Loops turn one command into an automation over any number of items.

© 2026 Tertiary Infotech Academy Pte Ltd

SCRIPT BUILDING BLOCKS 4

Functions, Exit Codes & Debugging

```
# define and call a function
$ greet() { local name=$1; echo "Hello, $name"; }    greet Linux
# every command returns an exit status
$ ls /nope; echo $?   # non-zero = failure
# set your script's own status
$ exit 1    # 0 = success
# trace execution line by line
$ bash -x script.sh    # or set -x / set +x inside
# clean up on Ctrl-C or exit
$ trap cleanup INT TERM EXIT
```

Functions structure bigger scripts; exit codes let scripts compose; -x and trap make them debuggable and robust.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Linux Shell Script

Labs 5–7

- Customising the Shell Environment
- Redirection, Pipes and Text Processing
- Your First Shell Script

Labs 8–10

- User Input, Arguments and Arithmetic
- Conditionals — if, test and case
- Loops — for, while and until

Labs 11–12

- Functions, Exit Status and Debugging
- Automation Script — Backup with tar

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Customising the Shell Environment

LAB 5

Configure your working environment: create aliases, set environment variables, extend PATH, customise the PS1 prompt, and persist the changes in ~/.bashrc.

You'll build

A personalised shell with a cls alias, a custom prompt and an extended PATH that survives new sessions.

Tools: bash, nano, env

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 5

```
# List your current aliases
$ alias

# Create aliases to fix a typo and clear the screen
$ alias grpe='grep' && alias cls='clear'

# Try the new aliases, then remove one
$ cls && unalias grpe

# Set an environment variable and read it back (no spaces …
$ MYNAME='Alfred' && echo $MYNAME

# Export it so child processes inherit it
$ export MYNAME && bash -c 'echo $MYNAME'

# Inspect the command search path
$ echo $PATH
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · VERIFY

Customising the Shell Environment

✅ Test it

After source ~/.bashrc, cls clears the screen, echo $PATH ends with your ~/bin, and the prompt shows user@host:directory.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Redirection, Pipes and Text Processing

LAB 6

Master the Unix I/O model: redirect stdout/stderr, pipe commands together, and process text with grep, regular expressions, sort, cut, uniq, wc, head, tail and diff.

You'll build

A processed fruit list — sorted, de-duplicated and pattern-matched — plus a saved process listing.

Tools: bash, grep, sort, cut, uniq, wc, diff

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 6

```
# Change into the lab folder with the exercise files
$ cd ~/course/labs/lab-06-text-processing

# Page through a long file with head and tail
$ head lorem_ipsum.txt && tail -3 lorem_ipsum.txt

# Redirect a sorted copy of fruit.txt into a new file …
$ sort fruit.txt > fruit_sorted.txt

# Append a new fruit to the file with >>
$ echo 'mango' >> fruit_sorted.txt

# Feed a file into a command with <
$ wc -l < fruit_sorted.txt

# Pipe sort into uniq to drop duplicates
$ sort fruit.txt | uniq | head
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · VERIFY

Redirection, Pipes and Text Processing

✅ Test it

grep -c apple fruit.txt returns a count, grep '^[sp]' lists only s/p fruits, processes.txt contains your ps aux snapshot, and the pipe chain prints unique sorted fruits.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Your First Shell Script

LAB 7

Write, save and execute your first Bash script: the #!/bin/bash shebang, echo output, variables, quoting rules and command substitution — then put it on your PATH.

You'll build

hello.sh and vars.sh — executable scripts that greet the user and report the date via command substitution.

Tools: bash, nano, chmod

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 7

```
# Create a scripts directory and change into it
$ mkdir -p ~/scripts && cd ~/scripts

# Create hello.sh with the shebang and an echo line
$ printf '#!/bin/bash\necho "Hello World from $(whoami)"\n' > hello.sh

# Try to run it — permission is denied because it is not …
$ ./hello.sh

# Make it executable and run it again
$ chmod +x hello.sh && ./hello.sh

# Create vars.sh using variables and quoting (single vs …
$ cp ~/course/labs/lab-07-first-script/vars.sh . && cat vars.sh

# Run vars.sh and study how $a, $b and $c interpolate inside …
$ bash vars.sh
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · VERIFY

Your First Shell Script

✅ Test it

./hello.sh prints the greeting with your username, vars.sh prints 'Hello, I have 16 apples', and hello.sh runs from any directory once it is in ~/bin.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

User Input, Arguments and Arithmetic

LAB 8

Make scripts interactive and reusable: read user input, use positional parameters $1..$9, $#, $@ and $0, and calculate with $(( )) arithmetic expansion.

You'll build

greet.sh (reads your name), args.sh (echoes its arguments) and calc.sh (adds two numbers).

Tools: bash, read

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 8

```
# Change into your scripts directory and copy the lab scripts
$ cd ~/scripts && cp ~/course/labs/lab-08-input-arguments/*.sh .
# Study greet.sh — read -p prompts the user and stores the …
$ cat greet.sh
# Run it and type your name when prompted
$ bash greet.sh
# Study args.sh — $0 is the script, $1/$2 the arguments, $# …
$ cat args.sh
# Run it with two arguments and check each expansion
$ bash args.sh apple banana
# Run it with five arguments — what changes?
$ bash args.sh a b c d e
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · VERIFY

User Input, Arguments and Arithmetic

✅ Test it

greet.sh greets you by the name you typed, args.sh reports the right count with $#, and calc.sh 15 27 prints 42.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Conditionals — if, test and case

LAB 9

Branch your scripts: if/elif/else with test brackets for numeric, string and file checks, and the case statement for multi-way selection.

You'll build

checknum.sh (positive/negative/zero), filecheck.sh (file tests) and fruitcase.sh (case menu).

Tools: bash, test

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 9

```
# Copy the lab scripts into your scripts folder
$ cd ~/scripts && cp ~/course/labs/lab-09-conditionals/*.sh .
# Study checknum.sh — if / elif / else with numeric tests …
$ cat checknum.sh
# Run it against different numbers
$ bash checknum.sh 5 && bash checknum.sh -3 && bash checknum.sh 0
# Study filecheck.sh — file tests: -e exists, -f regular …
$ cat filecheck.sh
# Test it against a file that exists and one that doesn't
$ bash filecheck.sh /etc/hostname && bash filecheck.sh /no/such/file
# Compare strings with = and != inside [ ]
$ s='linux'; if [ "$s" = 'linux' ]; then echo match; fi
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · VERIFY

Conditionals — if, test and case

✅ Test it

checknum.sh classifies 5, -3 and 0 correctly; filecheck.sh reports /etc/hostname as an existing regular file; fruitcase.sh hits its default branch for durian.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Loops — for, while and until

LAB 10

Automate repetition: for loops over lists, ranges and files; while and until loops with counters; and break/continue to control the flow.

You'll build

forloop.sh, countdown.sh (while) and a renamer one-liner that processes a batch of files.

Tools: bash

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 10

```
# Copy the lab scripts
$ cd ~/scripts && cp ~/course/labs/lab-10-loops/*.sh .
# Study forloop.sh — iterate a word list and a {1..5} range
$ cat forloop.sh
# Run it
$ bash forloop.sh
# Loop over files with a wildcard directly in the shell
$ for f in /etc/*.conf; do echo "config: $f"; done | head
# Study countdown.sh — a while loop with a counter and (( i-- …
$ cat countdown.sh
# Run the countdown
$ bash countdown.sh 5
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · VERIFY

Loops — for, while and until

✅ Test it

countdown.sh 5 counts 5-4-3-2-1 then 'Lift off!', the break/continue loop prints 1 2 4 5 6, and ~/batch holds report_1.bak … report_5.bak.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Functions, Exit Status and Debugging

LAB 11

Structure bigger scripts with functions, local variables and return values; read exit codes with $?; and debug with bash -x, set -x and trap.

You'll build

funcs.sh (function library), exitcodes.sh and a debugged script traced with set -x.

Tools: bash, trap

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 11

```
# Copy the lab scripts
$ cd ~/scripts && cp ~/course/labs/lab-11-functions/*.sh .

# Study funcs.sh — define a function, pass it arguments, keep …
$ cat funcs.sh

# Run it and watch each function fire
$ bash funcs.sh

# Check the exit status of a good and a bad command with $?
$ ls /etc > /dev/null; echo $? ; ls /nope 2>/dev/null; echo $?

# Study exitcodes.sh — return values and exit N set the …
$ cat exitcodes.sh && bash exitcodes.sh; echo "script exited with $?"

# Trace a script line by line with bash -x
$ bash -x funcs.sh
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · VERIFY

Functions, Exit Status and Debugging

✅ Test it

funcs.sh prints output from each function, a failed ls returns a non-zero $?, bash -x shows each command prefixed with +, and Ctrl-C on trapit.sh prints the cleanup message.

© 2026 Tertiary Infotech Academy Pte Ltd

Automation Script — Backup with tar

LAB 12

Put it all together: write backup.sh, a parameterised script that archives a directory with tar, gzip-compresses it, timestamps the filename, verifies the archive and reports disk usage.

You'll build

backup.sh — a reusable backup tool producing dated .tar.gz archives with a verification listing.

Tools: bash, tar, gzip, du, df

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 12

```
# Create some files worth backing up
$ mkdir -p ~/project && touch ~/project/app_{1..5}.conf && echo data > …
# Copy and study backup.sh — arguments, command substitution …
$ cd ~/scripts && cp ~/course/labs/lab-12-backup-script/backup.sh . && cat backup.sh
# Run it against the project directory
$ bash backup.sh ~/project
# List the archive contents to verify the backup (tar tzf)
$ tar tzf ~/backups/project-*.tar.gz | head
# Check the archive size and free disk space
$ du -h ~/backups && df -h / | tail -1
# Extract the archive into a restore directory to prove it …
$ mkdir -p ~/restore && tar xzf ~/backups/project-*.tar.gz -C ~/restore && ls …
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · VERIFY

Automation Script — Backup with tar

✅ Test it

A dated project-YYYY-MM-DD.tar.gz appears in ~/backups, tar tzf lists the five .conf files, the restore directory round-trips them, and a missing argument exits with status 1.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Linux Shell Script

- You can now: Configure the shell environment used to run integration scripts
- You can now: Select and combine text-processing tools for build and release pipelines
- You can now: Select scripts as the tool for integrating and deploying software
- You can now: Configure scripts that accept parameters for flexible deployment
- You can now: Configure scripts that branch on test results
- You can now: Configure scripts that repeat deployment steps over many items

© 2026 Tertiary Infotech Academy Pte Ltd

03

TOPIC 03

Linux Networking Configuration

Processes and cron · package management · deploying services · network configuration tests

© 2026 Tertiary Infotech Academy Pte Ltd

KEY CONCEPTS

Key Concepts — Linux Networking Configuration

1

Processes

ps, top and htop view processes; kill signals stop them; & , bg, fg and jobs manage background work.

2

Cron scheduling

cron + crontab automate scripts on a schedule — minute, hour, day, month, weekday.

3

Package managers

APT/dpkg on Debian-family, dnf/yum/rpm on Red Hat-family — install, upgrade, remove, autoremove.

4

Deploying services

Install a web server (NGINX/Apache), manage it with systemctl, verify it with curl.

5

Network configuration

ip addr, hostname, /etc/hosts, ping, ss — configure and inspect Linux networking.

6

Remote access & transfer

SSH for secure shells; scp/sftp for secure file transfer; diagnose issues from test results.

© 2026 Tertiary Infotech Academy Pte Ltd

Viewing Processes

```
# snapshot of your processes · everything
$ ps    ps aux
# full format · tree view
$ ps -ef    ps -e --forest
# one user's processes
$ ps -u username
# interactive monitors
$ top    htop    pstree
# top sorted by CPU for one user
$ top -n 10 -o %CPU -U root
```

ps aux is the everyday workhorse; top/htop watch the system live.

© 2026 Tertiary Infotech Academy Pte Ltd

JOB CONTROL & SIGNALS

Background & Foreground Jobs

1

command &

Start a command in the background.

2

Ctrl-C / Ctrl-Z

Kill / suspend the foreground process.

3

jobs

List this shell's jobs with their numbers.

4

bg %n · fg %n

Resume a suspended job in the background / bring it to the front.

5

kill PID

Send SIGTERM (15) — polite request to stop.

6

kill -9 PID

Send SIGKILL — force-stop when TERM is ignored.

© 2026 Tertiary Infotech Academy Pte Ltd

TIME-BASED JOB SCHEDULING

Cron — Scheduled Automation

```
# the five schedule fields
$ *  *  *  *  *   =  minute hour day-of-month month weekday

# every Monday at 07:00
$ 0 7 * * 1 /opt/sales/bin/weekly-report

# every minute (testing)
$ * * * * * $HOME/scripts/logdate.sh

# manage your schedule
$ crontab -l  list · crontab -e  edit · crontab -r  remove all

# shortcuts
$ @daily · @hourly · @reboot
```

cron runs your scripts on a schedule — redirect their output to a log file you can tail.

© 2026 Tertiary Infotech Academy Pte Ltd

Packages & Package Managers

Package

- **A package is**
  - A collection of files (the software) plus metadata
  - Description, version and dependencies
  - The unit of software installation on Linux

Manager

- **A package manager**
  - Installs, upgrades and removes packages
  - Resolves and manages dependencies
  - Tracks what is installed
  - RPM distros: dnf/yum/rpm · DEB distros: apt/dpkg

© 2026 Tertiary Infotech Academy Pte Ltd

APT — Debian / Ubuntu

```
# refresh the local package list first
$ sudo apt update

# search and inspect
$ apt search nginx    apt show nginx

# install · remove · purge configs too
$ sudo apt install -y nginx    sudo apt remove nginx    sudo apt purge nginx

# upgrade everything installed
$ sudo apt upgrade -y

# clean unused dependencies
$ sudo apt autoremove

# the low-level tool
$ dpkg -l · dpkg -L pkg · dpkg -i pkg.deb
```

apt combines the older apt-get and apt-cache; dpkg works on .deb files directly.

© 2026 Tertiary Infotech Academy Pte Ltd

dnf / yum / rpm — Red Hat Family

```
# RHEL 8+ / CentOS 8+ / Fedora
$ sudo dnf install -y package    dnf search string
# RHEL 7 / CentOS 7 and earlier
$ sudo yum install -y package
# info · remove · upgrade
$ dnf info pkg    dnf remove pkg    dnf upgrade
# the low-level tool
$ rpm -qa list · rpm -ql pkg files · rpm -ivh pkg.rpm install
# keep it clean
$ dnf autoremove
```

Same concepts as APT — different family, different commands.

© 2026 Tertiary Infotech Academy Pte Ltd

A REAL RELEASE PIPELINE

Deploying a Web Server

1. sudo apt update — refresh package lists
2. sudo apt install -y nginx — install the release
3. service nginx start · systemctl status — run and check the service
4. Publish content to /var/www/html
5. curl -I http://localhost — configuration test: expect HTTP 200

© 2026 Tertiary Infotech Academy Pte Ltd

TAR · GZIP · DF · DU

Archiving & Disk Usage

```
# compress · decompress
$ gzip shakespeare.txt    gunzip shakespeare.txt.gz
# create a compressed archive
$ tar czf backup.tar.gz mydir/
# list an archive's contents (verify!)
$ tar tzf backup.tar.gz
# extract
$ tar xzf backup.tar.gz    # c create · x extract · t list · v verbose · z gzip · f file
# estimate directory usage · free space
$ du -h mydir    df -h
```

tar bundles a directory tree into one file; gzip compresses it — the classic release artifact.

© 2026 Tertiary Infotech Academy Pte Ltd

Copying Files over the Network

1

scp

Secure copy: scp source user@host:dest — one-shot transfers over SSH.

2

sftp

Interactive secure file transfer session: sftp user@host.

3

ssh

Secure remote shell — the admin's front door to every server.

4

Windows clients

PuTTY (pscp/psftp, PuTTYgen for key conversion), WinSCP, FileZilla, Cyberduck.

5

Keys over passwords

SSH keys authenticate without passwords — generate, convert, protect them.

6

Legacy ftp

Unencrypted — replace with sftp/scp wherever you find it.

© 2026 Tertiary Infotech Academy Pte Ltd

CONFIGURE · TEST · DIAGNOSE

Network Configuration & Tests

```
# interfaces and addresses
$ ip addr
# routing table / default gateway
$ ip route
# reachability — by IP, then by name
$ ping -c 4 8.8.8.8    ping -c 4 google.com
# local name mapping
$ echo '127.0.0.1 myapp.local' | sudo tee -a /etc/hosts
# listening sockets
$ ss -tuln
# layer-7 test
$ curl -I http://localhost
```

Test in layers: interface → gateway → IP reachability → DNS → application. Each failure isolates the fault.

© 2026 Tertiary Infotech Academy Pte Ltd

DIAGNOSIS FROM RESULTS

Interpreting Test Results

1

No IPv4 address

Interface or DHCP problem — check ip addr and the network config.

2

Gateway unreachable

Local network problem — check ip route, cabling, the VM network.

3

IP ping OK, name ping fails

DNS problem — check /etc/resolv.conf and /etc/hosts.

4

HTTP not 200

Service problem — is it running? Listening on the port? Firewall blocking?

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Linux Networking Configuration

Labs 13–13

- Managing Processes and Scheduling with Cron

Labs 14–14

- Package Management and Web Server Deployment

Labs 15–15

- Network Configuration and Troubleshooting

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Managing Processes and Scheduling with Cron

LAB 13

Inspect and control processes with ps, top and kill; run jobs in the background with &, bg and fg; then automate a script with a cron schedule and verify it from its log.

You'll build

A cron job that runs logdate.sh every minute and appends timestamps to ~/cron.log.

Tools: ps, top, kill, cron, crontab

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 13

```
# List your processes, then everything on the system
$ ps && ps aux | head

# Watch processes interactively with top (press q to quit)
$ top -n 1 | head -15

# Start a long-running background job with &
$ sleep 600 &

# List jobs and find the sleep's process ID
$ jobs && ps aux | grep 'sleep 600' | grep -v grep

# Kill the background process by PID (old-deck exercise)
$ kill $(pgrep -f 'sleep 600') && jobs

# Open a second bash shell, find its PID with ps aux | grep …
$ bash & ps aux | grep bash | head && kill %+
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · VERIFY

Managing Processes and Scheduling with Cron

✅ Test it

The killed sleep disappears from jobs, and ~/cron.log gains one timestamp line per minute while the crontab entry is active.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Package Management and Web Server Deployment

LAB 14

Manage software with APT and dpkg, then deploy a real service: install NGINX, publish a page, control the service, and run a configuration test with curl to interpret the result.

You'll build

A running NGINX web server serving your own index.html, verified by an HTTP 200 from curl.

Tools: apt, dpkg, nginx, systemctl/service, curl

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 14

```
# Refresh the package lists (always do this first)
$ sudo apt update

# Search the repositories and inspect a package before …
$ apt search nginx 2>/dev/null | head && apt show nginx | head

# Install the NGINX web server
$ sudo apt install -y nginx

# Confirm what dpkg installed
$ dpkg -l | grep nginx

# Start the service and check its status
$ sudo service nginx start && service nginx status

# Run a configuration test: request the default page and read …
$ curl -I http://localhost
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · VERIFY

Package Management and Web Server Deployment

✅ Test it

curl -I returns HTTP/1.1 200 OK and curl http://localhost shows your deployed heading; nginx -t reports the configuration file syntax is ok.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Network Configuration and Troubleshooting

LAB 15

Inspect and test Linux networking: interfaces and addresses with ip, name resolution with /etc/hosts, reachability with ping, sockets with ss — then run netcheck.sh to automate the diagnosis and interpret its report.

You'll build

netcheck.sh — a network health-check script that tests interface, DNS, gateway and web reachability and prints a PASS/FAIL report.

Tools: ip, ping, ss, getent, curl, ssh/scp (concept)

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 15

```
# List the network interfaces and their IP addresses
$ ip addr
# Show the routing table and identify the default gateway
$ ip route
# Check the hostname and its resolution
$ hostname && getent hosts $(hostname)
# Test reachability with ping (4 packets)
$ ping -c 4 8.8.8.8
# Test name resolution + reachability together
$ ping -c 4 google.com
# Map a friendly name in /etc/hosts and ping it
$ echo '127.0.0.1 myapp.local' | sudo tee -a /etc/hosts && ping -c 2 myapp.local
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · VERIFY

Network Configuration and Troubleshooting

✅ Test it

ping 8.8.8.8 and ping google.com both succeed, ss -tuln lists the listening services, and netcheck.sh prints PASS for interface, gateway, DNS and web tests.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Linux Networking Configuration

- You can now: Execute configuration tests on running processes and schedule automated jobs
- You can now: Configure software products and deploy a release to the platform, then test it
- You can now: Execute network configuration tests and diagnose issues from the results

© 2026 Tertiary Infotech Academy Pte Ltd

04

TOPIC 04

Linux Security

Security principles · least privilege · firewall · hardening and audit scripts

© 2026 Tertiary Infotech Academy Pte Ltd

KEY CONCEPTS

Key Concepts — Linux Security

1

Security trade-offs

Nothing is perfectly secure — balance convenience against risk; assess severity, probability and cost.

2

Multiuser model

Every file and process has an owner; compromising one account need not compromise the system.

3

Least privilege

Use root only when required; avoid shared accounts and direct root logins; control access with sudo.

4

Minimise & encrypt

Install only what you need; replace FTP/telnet/HTTP with SFTP/SSH/HTTPS.

5

Firewall & monitoring

Filter connections with ufw/iptables; review logs and monitor system activity routinely.

6

Continuous improvement

Audit configurations with scripts, identify improvements and implement modifications.

© 2026 Tertiary Infotech Academy Pte Ltd

IS LINUX SECURE?

Nothing is perfectly secure — security is a series of trade-offs.

No passwords = easy but insecure. Powered off = secure but unusable. Linux is only as secure as you make it.

© 2026 Tertiary Infotech Academy Pte Ltd

FOUR QUESTIONS BEFORE EVERY CONTROL

Risk Assessment

1. What is the severity of the risk?
2. What is the probability of it occurring?
3. What is the cost to mitigate it?
4. How effective is the countermeasure?

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURAL ADVANTAGES

Why Linux Has a Security Edge

Multiuser

- **Multiuser by design**
  - Every file and every process has an owner
  - Permissions contain a compromised account
  - Only root can do anything — protect it

Open source

- **Open + centrally managed**
  - Open source: no black box, flaws get found
  - Package repositories: tested, signed software
  - One command updates everything installed
  - Windows: third-party downloads, untested code

© 2026 Tertiary Infotech Academy Pte Ltd

HARDENING PRINCIPLES 1

Security Best Practices — Reduce the Surface

1

Minimise software

If you don't need it, don't install it; uninstall what you stop using.

2

Minimise services

If you don't need a service, don't start it — every listener is attack surface.

3

Separate systems

One compromised service shouldn't take the others with it.

4

Encrypt in transit

FTP→SFTP, telnet→SSH, HTTP→HTTPS, SNMPv1/2→v3 — defeat eavesdropping.

© 2026 Tertiary Infotech Academy Pte Ltd

HARDENING PRINCIPLES 2

Security Best Practices — Control Access

1

Least privilege

Use root only when required; restrictive permissions that still let people work.

2

No shared accounts

One person, one account — shared accounts destroy accountability.

3

No direct root logins

Log in as yourself, then sudo — controlled and monitored.

4

Multifactor authentication

Something you know + something you have or are (password + OTP/fingerprint).

5

Maintain accounts

Have a process to remove access when people leave.

6

Monitor activity

Review logs routinely; send them to a central logging system.

© 2026 Tertiary Infotech Academy Pte Ltd

TECHNICAL CONTROLS

The Firewall & Data at Rest

```
# Linux ships a firewall: netfilter + iptables/ufw
$ sudo ufw allow ssh && sudo ufw enable
# verify the rules
$ sudo ufw status verbose
# see what is exposed before and after
$ ss -tuln
# lock down a sensitive file
$ chmod 600 ~/secret.key
# find risky world-writable files
$ find / -type f -perm -o+w
# encrypt data at rest
$ Full-disk or per-file encryption protects data on disk
```

Only allow connections from desired sources; encryption protects data even if the disk walks away.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Linux Security

Labs 16–16

- Hardening Linux Security

Labs 17–17

- Building a Security Audit Script

—

- —

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON

Hardening Linux Security

LAB 16

Apply the security principles: enforce least privilege on files, lock down SSH-style access, review accounts, and enable the ufw firewall so only the services you need are reachable.

You'll build

A hardened lab machine — private key file at 600, firewall enabled with only SSH/HTTP allowed, and a reviewed account list.

Tools: chmod, passwd, ufw, ss, sudo

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 16

```
# Review the accounts on the system — real users have a …
$ cut -d: -f1,7 /etc/passwd | head -15
# Confirm only root has UID 0 (any other UID-0 user is a red …
$ awk -F: '$3==0 {print $1}' /etc/passwd
# Create a sensitive file and apply least privilege — …
$ touch ~/secret.key && chmod 600 ~/secret.key && ls -l ~/secret.key
# Find world-writable files in your home tree — each is a …
$ find ~ -maxdepth 2 -type f -perm -o+w 2>/dev/null
# Check which services are listening before the firewall goes …
$ ss -tuln
# Install and enable the ufw firewall, allowing only SSH and …
$ sudo apt install -y ufw && sudo ufw allow ssh && sudo ufw allow http && sudo ufw …
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · VERIFY

Hardening Linux Security

✅ Test it

ls -l shows -rw------- on secret.key, ufw status reports active with only 22 and 80 allowed, and the UID-0 check returns root alone.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON

Building a Security Audit Script

LAB 17

Combine everything you learned — functions, conditionals, loops, pipes and text processing — into audit.sh, a security audit script that checks accounts, permissions, listening services and disk usage, then writes a timestamped report.

You'll build

audit.sh — a security audit tool producing ~/audit-YYYY-MM-DD.txt with a finding per check.

Tools: bash, awk, find, ss, df

© 2026 Tertiary Infotech Academy Pte Ltd

Key Commands — Lab 17

```
# Copy and study audit.sh — one function per check, a report …
$ cp ~/course/labs/lab-17-security-audit/audit.sh ~/scripts/ && cat …

# Run the audit
$ bash ~/scripts/audit.sh

# Read the report it generated
$ cat ~/audit-*.txt

# Fix a finding — remove world-write from any flagged file — …
$ chmod o-w <flagged-file> && bash ~/scripts/audit.sh

# Extend the script: add a check that the ufw firewall is …
$ sudo ufw status | grep -q 'Status: active' && echo PASS || echo FAIL

# Schedule the audit to run daily at 07:00 with cron — …
$ ( crontab -l 2>/dev/null; echo '0 7 * * * $HOME/scripts/audit.sh' ) | crontab - …
```

The detailed step-by-step walkthrough is in the Learner Guide and labs/ folder.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 17 · VERIFY

Building a Security Audit Script

✅ Test it

audit.sh writes a dated report listing each check with PASS/FAIL, the re-run shows your fix flipped a FAIL to PASS, and crontab -l shows the daily 07:00 entry before cleanup.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Linux Security

- You can now: Identify potential improvements to the Linux security configuration and implement them
- You can now: Implement a script that audits the configuration and surfaces improvements

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES ACHIEVED

What You Achieved

1

Linux Fundamental

Analysed Linux release components, the file system, users and permissions.

2

Shell Scripting

Built scripts with variables, input, conditionals, loops, functions and debugging.

3

Networking Configuration

Managed processes and cron, deployed NGINX with APT, tested and diagnosed the network.

4

Linux Security

Applied least privilege, enabled the firewall and automated a security audit.

© 2026 Tertiary Infotech Academy Pte Ltd

TEST YOURSELF

Practice Exam

- Sharpen your readiness with the Tertiary Infotech practice exams portal.
- Practice exams: https://exams.tertiaryinfotech.com
- Attempt the Linux practice questions under timed conditions and review every explanation.
- Revisit any lab whose topic you miss.

© 2026 Tertiary Infotech Academy Pte Ltd

KEEP LEARNING

Recommended Next Courses

1. CompTIA Linux+ Training
2. Cyber Security Awareness Workshop
3. WordPress Masterclass
4. AI Devops with Docker
5. Kubernetes and Cloud Native Associate (KCNA) Training
6. Browse all courses at www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd

WE'RE HERE TO HELP

Support

- If you have any enquiries during and after the class, contact us:
- Email: enquiry@tertiaryinfotech.com
- Tel: +65 6100 0613
- Website: www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE FEEDBACK

Course Feedback & Certificate

- Share your feedback in the course survey on the LMS — it helps us improve the course.
- Your certificate of completion is issued on the LMS at the end of the course.
- Portal: https://lms-tms.tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

END OF COURSE

Thank You!

You can now configure Linux, automate it with shell scripts, and secure it — happy scripting!

© 2026 Tertiary Infotech Academy Pte Ltd